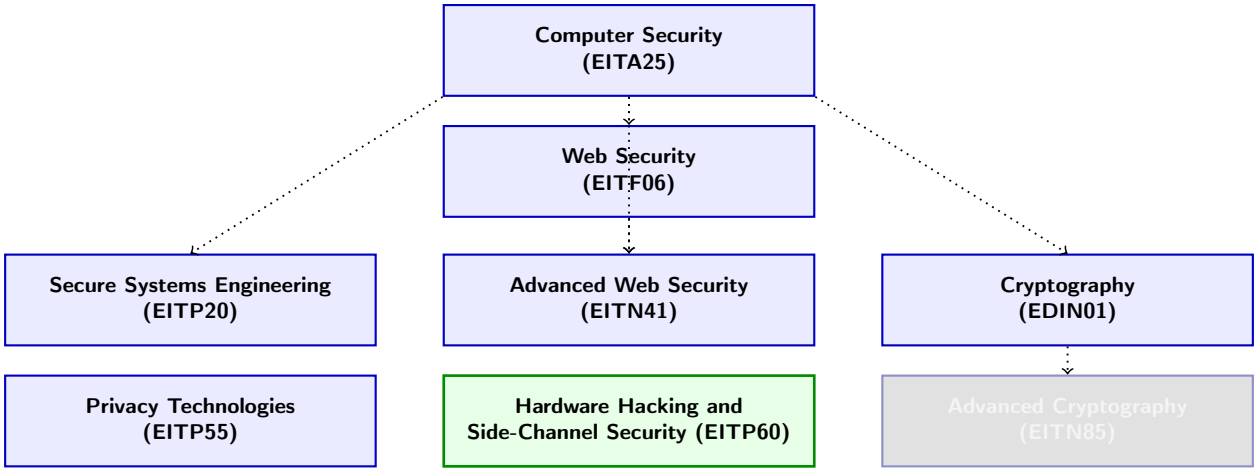
Hardware Hacking and Side-Channel Security

Course Project: Power Side-Channel Analysis with ELMO

- The project is normally completed in groups of two students. A one-student project is allowed only in a very special case and requires prior approval from the instructor.
- The project has two parts: first learn and validate ELMO; then select a paper using an unprotected Cortex-M0/M4 implementation, replace its laboratory traces with ELMO traces, and repeat the analysis.
- Each group, or an approved solo student, gives a five-minute presentation of its work to the instructor.
- The report should be approximately five pages. A correct reproduction is required; substantial, rigorous work beyond the paper is normally expected for grade 5.
- You will not receive detailed instructions for every step. Choosing a sound experimental procedure and documenting it clearly are part of the project.

Learning goals:

- Learn how ELMO models power leakage from ARM software execution.
- Generate, inspect, and analyze simulated side-channel traces.
- Translate a real Cortex-M0/M4 experiment into a reproducible simulation workflow.
- Compare simulated leakage with the paper's physical power or EM measurements.
- Communicate experimental methods, limitations, and results concisely.



General Information

The purpose of this project is to connect a published **real-device** side-channel result with a reproducible simulation experiment. The selected paper must run cryptographic software on a real Cortex-M0 or Cortex-M4 and analyze physical power or EM traces. Instead of collecting those laboratory traces with an oscilloscope or probe, you will run the corresponding public ARM code in **ELMO**, generate simulated traces, and repeat the paper's analysis. The project is normally completed in **groups of two students** and consists of the two parts below. A student may work alone only in a very special case and after receiving approval from the instructor.

Your result does not need to match the paper's physical measurement exactly. ELMO models a Cortex-M0 and cannot reproduce every property of a real device, especially a Cortex-M4. You are expected to explain what was reproduced, what differed, and whether each difference follows from the leakage model, instruction set, microarchitecture, measurement noise, or your experimental choices.

Failure policy. Successful leakage detection or key recovery is not mandatory when ELMO does not reproduce the class separation observed on the physical device. A negative result is acceptable only if the group verifies functional correctness with test vectors, validates trace generation and alignment, applies the intended analysis with justified parameters and controls, and investigates likely causes. Incomplete code, broken analysis, or an unjustified number of traces is not an acceptable negative result.

1 Part 1: Learn ELMO

The goal of Part 1 is to establish a working and understood ELMO workflow before starting the paper reproduction. Follow the course ELMO quick-start material and complete the following tasks:

1. Install or obtain the required ARM toolchain and ELMO environment.
2. Compile a provided example for the ARM target used by ELMO.
3. Generate several simulated power traces and identify the code region delimited by the trace triggers.
4. Inspect the trace format and explain what the trace samples represent.
5. Perform at least one basic analysis supplied in the course material, such as Welch's t-test (TVLA) or correlation power analysis (CPA).
6. Save one figure or table that demonstrates that your ELMO and analysis workflow operates correctly.

Part 1 is a warm-up, but it must be documented briefly in the final report. Include the build and execution commands, the ELMO configuration, the number of traces, and the validation result. Every student should be able to explain the entire workflow.

2 Part 2: Reproduce a Published Result

Select **one of the two papers below**. Each paper reports a side-channel experiment using a real Cortex-M0 or Cortex-M4 and has public implementation or reproduction support on GitHub. The core substitution in this project is:

Paper: public ARM code → real Cortex-M0/M4 → physical power/EM traces → analysis
Project: corresponding ARM code → ELMO → simulated power traces → the same analysis → comparison

Do not try to reproduce an entire paper. Select one operation and one clearly identified leakage test, attack result, figure, table, or claim. **Use only an unprotected, non-masked implementation.**

Option 1—CPA on Elephant-Dumbo

L. Vialar, "Trumping the Elephant: Fast Side-Channel Key-Recovery Attack against Dumbo," *SSTIC*, 2022.
Paper: IACR ePrint 2022/446
GitHub attack repository: `kudelskisecurity/nist-lwc-power-analysis`
Real experiment: the paper uses CPA on physical power traces from an unprotected Elephant-Dumbo implementation running on an STM32F303 Cortex-M4. It attacks the first Spongent round during absorption of the first associated-data block. The GitHub repository contains the target sources, ChipWhisperer support, leakage models, complete attack code, and usage instructions.
Simple ELMO target: run only the targeted first Spongent round, generate ELMO traces with chosen associated data, and reproduce CPA for one or a few mask bytes. Full-key recovery and the paper's exhaustive-search correction step are not required.

Option 2—Chosen-ciphertext leakage attack on Kyber

P. Ravi, S. Sinha Roy, A. Chattopadhyay, and S. Bhasin, "Generic Side-channel Attacks on CCA-secure Lattice-based PKE and KEMs," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2020(3), pp. 307–335.
Paper: IACR ePrint 2019/948
GitHub attack repository: code, traces, and attack scripts
Real experiment: the paper uses physical EM traces from an unprotected Kyber512 implementation on an STM32F4 Cortex-M4. Leakage from the decrypted message is converted into a binary plaintext-checking oracle and then used for key recovery. The authors' GitHub repository contains the modified target code, ciphertext-generation scripts, example traces, and attack scripts.
Simple ELMO target: reproduce only the attack on one secret coefficient. Use the five chosen-ciphertext cases given for Kyber512, generate ELMO traces, distinguish the two oracle classes by a trace mean, difference, or simple classifier, and infer the coefficient. Full-polynomial and full-key recovery are not required.

Cortex-M4 porting rule

ELMO models Cortex-M0 software; it is not a Cortex-M4 simulator. For Cortex-M4 papers, select a small C routine and compile or translate it to functionally equivalent Cortex-M0 code. Avoid M4-specific assembly. State any code changes and verify the routine with test vectors before generating traces.

For any option, the group must state exactly which paper result, repository commit, test implementation, and paper figure/table/claim it targets. Confirm that the selected experiment builds early; if an old dependency causes a problem, document the compatible environment rather than silently replacing the authors' code.

Required experimental work

For the selected paper, your group must:

1. choose one paper result and one small target operation;
2. obtain the public code, record the commit hash, and make the operation run correctly in ELMO;
3. generate ELMO traces with documented inputs and trigger boundaries;
4. apply the same basic analysis as the paper, such as CPA, TVLA, or trace comparison; and
5. compare the simulated result with the physical result and explain important differences.

Clearly label which code originates from the authors and which code or scripts were written or modified by your group. Do not claim that a result from an ELMO trace is a physical measurement.

3 Assessment and Work Beyond Reproduction

A faithful and correctly explained reproduction is the **core requirement**. For a higher score, the group should go beyond merely re-running the authors' commands. In particular, **substantial, well-designed, and correctly evaluated work beyond the paper is normally expected for grade 5**. Examples include:

- repeating the analysis for another unprotected implementation or input class;
- running a justified trace-count, artificial-noise, or statistical-power study;
- testing how compiler options or alternative M4-to-M0 ports change leakage;
- extending CPA to additional key bytes or coefficients;
- comparing two compiler settings or simple source-code variants; or
- investigating a new question suggested by, but not answered in, the paper.

More traces or more code do not automatically give a higher grade. The extension must have a clear question, appropriate controls, reproducible evidence, and a careful interpretation. In the report and presentation, label results clearly as **reproduced** or **new/extended**.

4 Report

Each group submits one joint report of **approximately five pages**, excluding references and an optional appendix. An approved solo student submits one individual report. The report should be concise and reproducible. It must contain:

1. **Team and target:** all group members' names, the selected paper, its DOI or stable link, the GitHub repository URL, and the tested commit hash.
2. **Part 1:** a short account of the ELMO setup and the validation result.
3. **Part 2 method:** the paper's physical experiment, your target code, any M4-to-M0 changes, ELMO settings, inputs, number of traces, and analysis.

4. **Results:** readable figures or tables with labelled axes, units, parameters, and captions.
5. **Comparison and discussion:** compare with the paper, explain limitations of simulation, and distinguish reproduced from new results.
6. **Reproduction and contributions:** give the commands needed to recreate the result and briefly state each member's contribution.

Reference all reused code, text, data, and figures. Normal academic-integrity rules apply. The report must be submitted as a PDF.

5 Code and Reproducibility Package

Submit the source code and scripts required to reproduce your results. The package must include a short `README` and should use relative paths. Do not submit only generated traces or figures. At minimum, include:

- the target C/assembly source and build files;
- ELMO input generation and configuration files;
- analysis and plotting scripts;
- exact commands for a clean reproduction; and
- a note identifying upstream code and your modifications.

Large generated trace files may be omitted if your scripts can regenerate them. Your report and code should use fixed seeds where randomness is involved, or state clearly when exact numerical repetition is not expected.

6 Five-Minute Presentation

Each group, or an approved solo student, will give a **five-minute presentation to the instructor**. All group members must attend and be prepared to answer questions. Aim for approximately four or five slides:

1. the selected paper, its real Cortex-M0/M4 setup, and the targeted result;
2. how you replaced its physical traces with ELMO traces;
3. the most important simulated result, compared directly with the paper's physical result;
4. limitations, lessons learned, and any additional observation; and
5. the division of work, if it is not already clear.

Keep setup details brief and spend most of the time explaining the result. The presentation may add one result from your extension, but it must still fit within five minutes. The presentation is part of the assessment; slides alone are not a substitute for the oral presentation.

7 Submission Checklist

Before submitting, verify that your group has included:

- one report PDF (approximately five pages, excluding references/appendix);
- one complete code and reproducibility package with a `README`;
- presentation slides for a five-minute presentation;
- the exact paper citation, GitHub URL, and commit hash; and
- all group members' names and a contribution statement.

Deadlines, presentation times, and the submission channel will be announced on the course platform.